

1. Content

1. [Content](#)
2. [Summary](#)
3. [Motivations](#)
4. [Description of the project](#)
5. [Requirements](#)
6. [The Full System](#)
7. [Preview of the project architecture](#)
8. [Overview of the project's structure](#)
9. [Layer diagram with modules](#)
10. [Project File Structure](#)
11. [The Simulated Scenarios](#)
12. [Implementation Order — Step by Step](#)

2. Summary

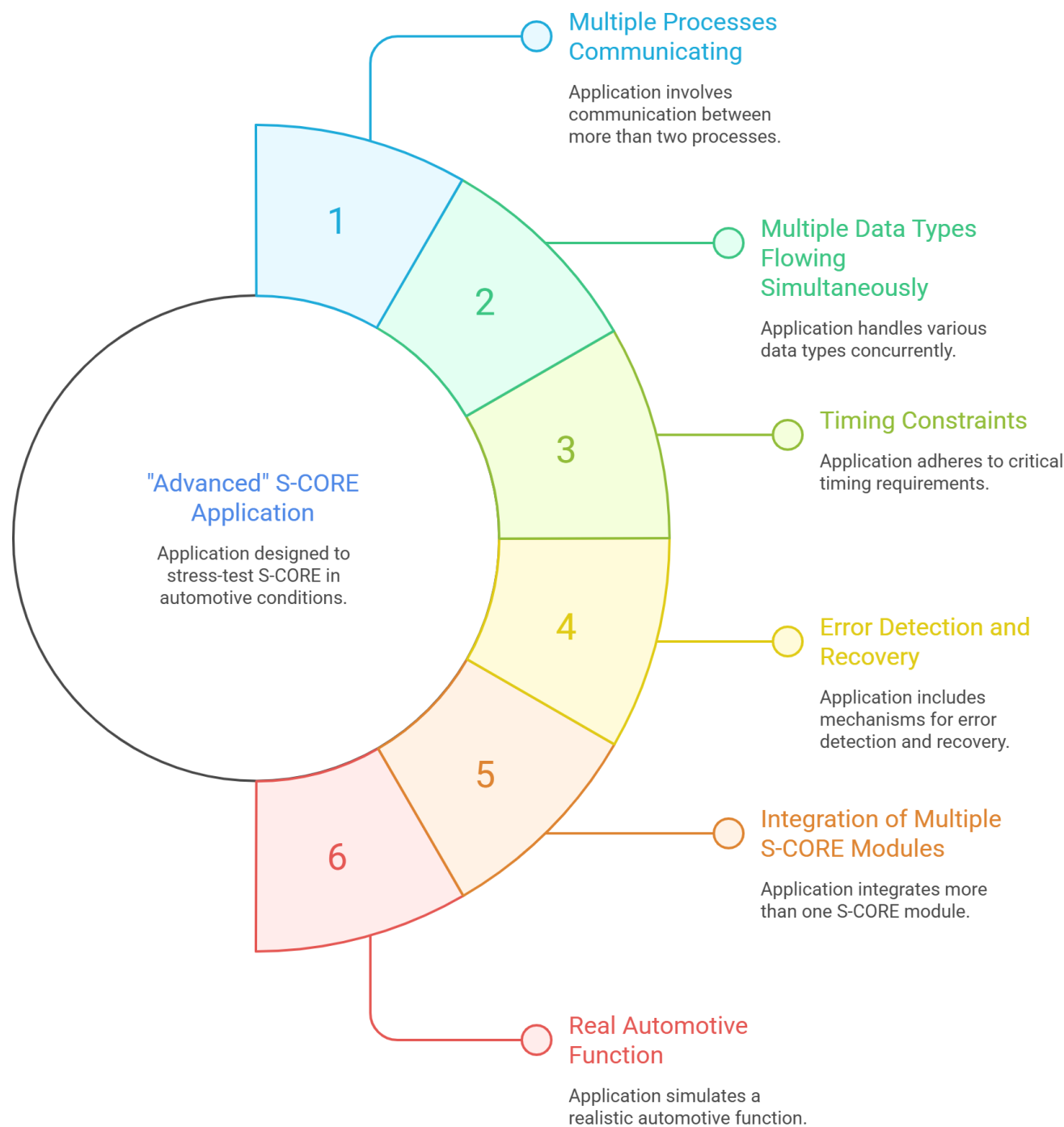
This document describes the design of an ADAS Pipeline application built on Eclipse S-CORE middleware. The application implements Cruise Control and Automatic Emergency Braking (AEB) across 9 processes organized in 6 layers, integrating the COM, FEO, LOG, PERSISTENCE and EXEC modules. The goal is to stress-test S-CORE under realistic automotive conditions and evaluate its readiness for deployment on embedded hardware such as the NXP S32N7 with QNX.

3. Motivations

In this project we intend to go beyond the basic examples and tests of the eclipse s-core and focus more on, but rather see if the Eclipse S-CORE can work as a middleware for a real embedded application, and if it is ready for deployment on hardware (like the NXP S32N7 + QNX). This means our next application shouldn't just be a demo — it should stress-test S-CORE in conditions that resemble real automotive use:

- ✓ Multiple processes communicating (not just 2)
- ✓ Multiple data types flowing simultaneously
- ✓ Timing constraints (cycle times that matter)
- ✓ Error detection and recovery
- ✓ Integration of more than one S-CORE module
- ✓ Something that resembles a real automotive function

Unveiling the Dimensions of a "advanced" S-CORE Application



Made with  Napkin

4. Description of the project

In this project we want to build an application that includes some ADAS functionalities like Cruise control and Automatic Emergency Braking (AEB), in which we want to use the eclipse s-core platform as a middleware.

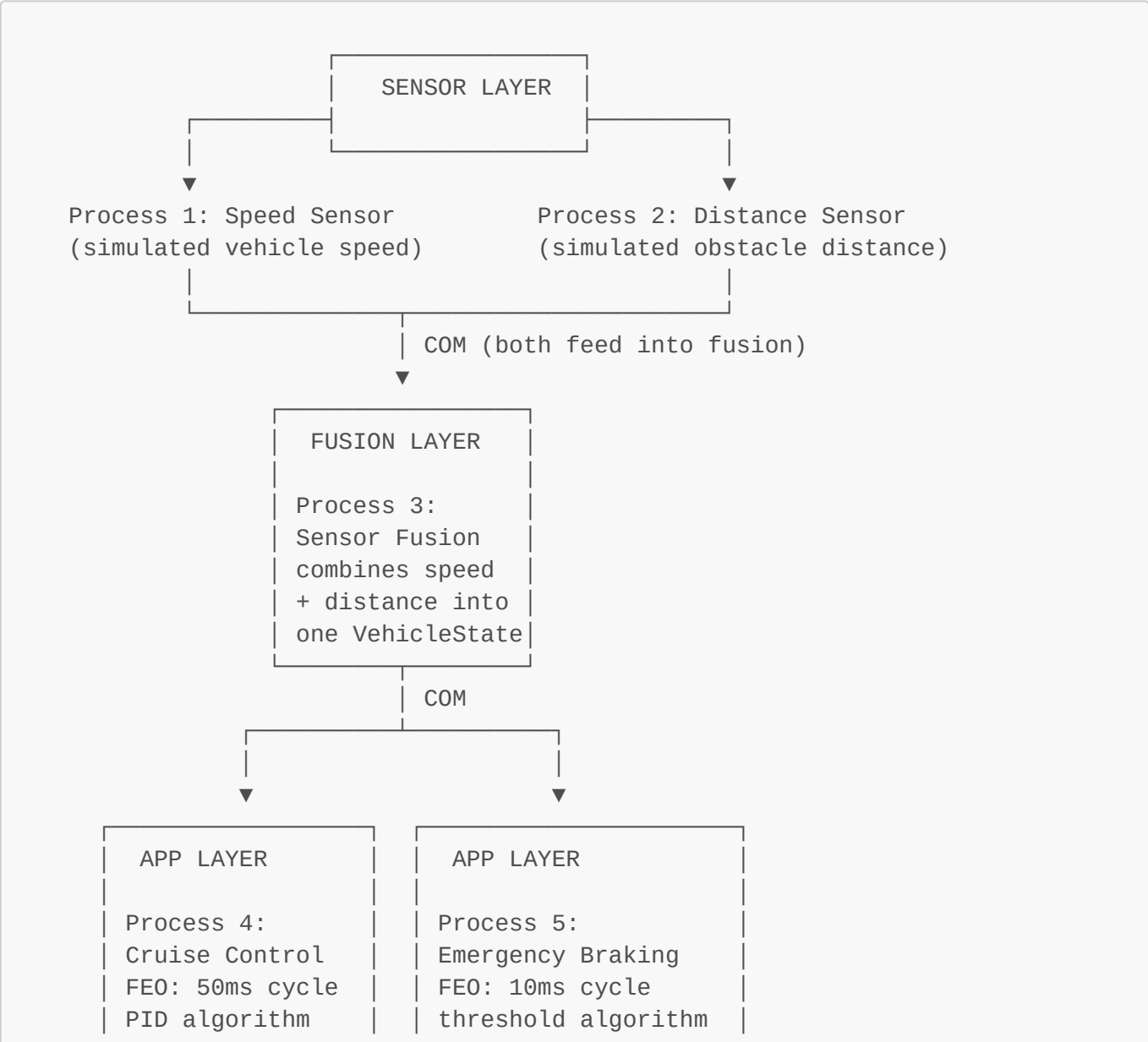
5. Requirements

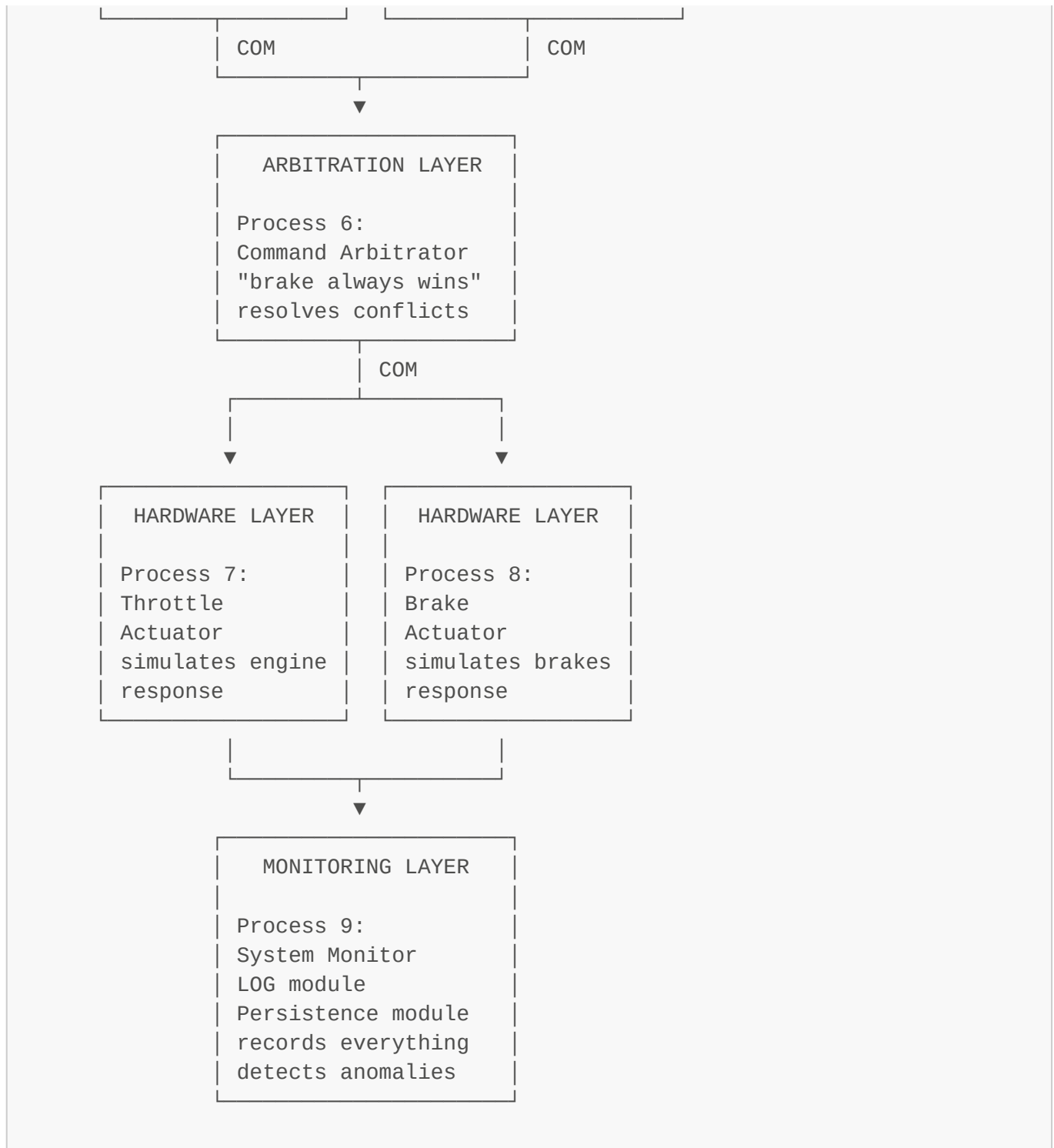
- Functional Requirements:
- FR-1: The system shall simulate vehicle speed from 0 to 120 km/h
 - FR-2: Cruise control shall maintain target speed within ± 2 km/h
 - FR-3: Emergency braking shall activate when obstacle distance < 30m
 - FR-4: Brake command shall always override throttle command
 - FR-5: System monitor shall detect and log missed ACKs
- Non-Functional Requirements:
- NFR-1: Emergency brake process cycle time shall not exceed 10ms
 - NFR-2: All inter-process communication shall use Eclipse S-CORE COM
 - NFR-3: All events shall be logged with timestamps
 - NFR-4: System shall run on Linux (target: portable to QNX)

6. The Full System

Before the details, here is the complete picture in one diagram:

7. Preview of the project architecutre





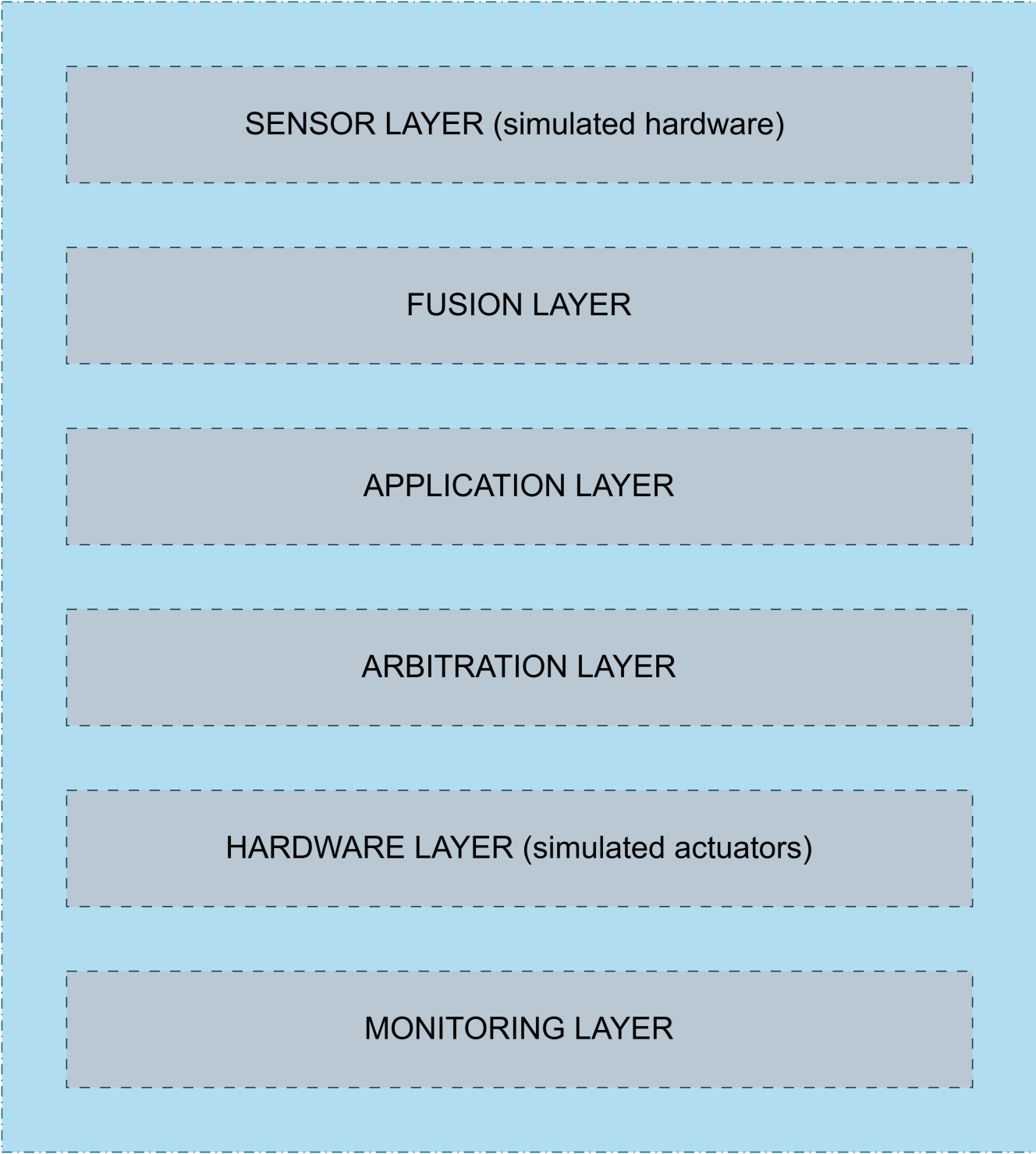
The layered architecture was deliberately chosen to reflect real automotive software stacks. As a result:

"We structured the application to mirror the real AUTOSAR Adaptive layered architecture, so the S-CORE COM channels correspond to the inter-ECU communication paths we would use on the NXP S32N7."

Sensor Layer	→ Hardware Abstraction Layer (HAL)
Fusion Layer	→ Sensor fusion ECU
Application Layer	→ ADAS functions (cruise, AEB)
Arbitration Layer	→ Safety arbitration / supervisory controller

Hardware Layer → Actuator ECUs (throttle, brake ECUs)
Monitoring Layer → Diagnostic and logging ECU

8. Overview of the project's structure



SENSOR LAYER (simulated hardware)

Process 1: speed_sensor

Simulates vehicle speed (0 → 120 km/h, realistic ramp)
Skeleton on: score/SpeedSensor
FEO cycle: 20ms

Process 2: distance_sensor

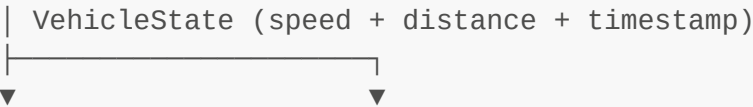
Simulates obstacle distance (100m → 2m, obstacle approaching)
Skeleton on: score/DistanceSensor
FEO cycle: 20ms



FUSION LAYER

Process 3: sensor_fusion

Proxy on: score/SpeedSensor + score/DistanceSensor
Combines both into one VehicleState struct
Skeleton on: score/VehicleState
FEO cycle: 20ms (runs after both sensors)



APPLICATION LAYER

Process 4: cruise_control

Proxy on: score/VehicleState
PID algorithm:
if speed < target → throttle
if speed > target → release
Skeleton on: score/CruiseCmd
FEO cycle: 50ms

Process 5: emergency_brake

Proxy on: score/VehicleState
Threshold algorithm:
if distance < 30m → brake
force = f(distance, speed)
Skeleton on: score/BrakeCmd
FEO cycle: 10ms (safety critical)



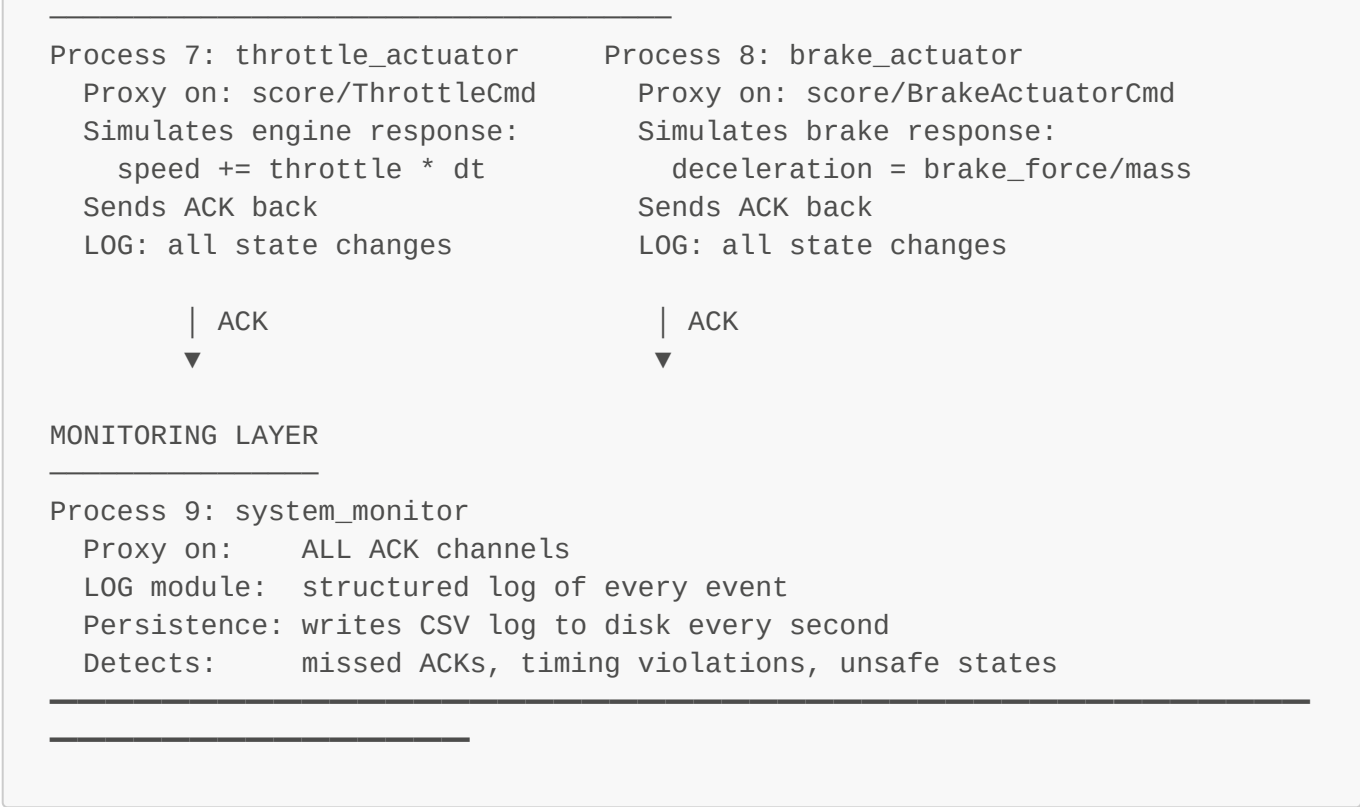
ARBITRATION LAYER

Process 6: command_arbitrator

Proxy on: score/CruiseCmd + score/BrakeCmd
Rule: brake ALWAYS overrides throttle
Rule: if brake active → send throttle = 0
Skeleton on: score/ThrottleCmd + score/BrakeActuatorCmd
FEO cycle: 10ms

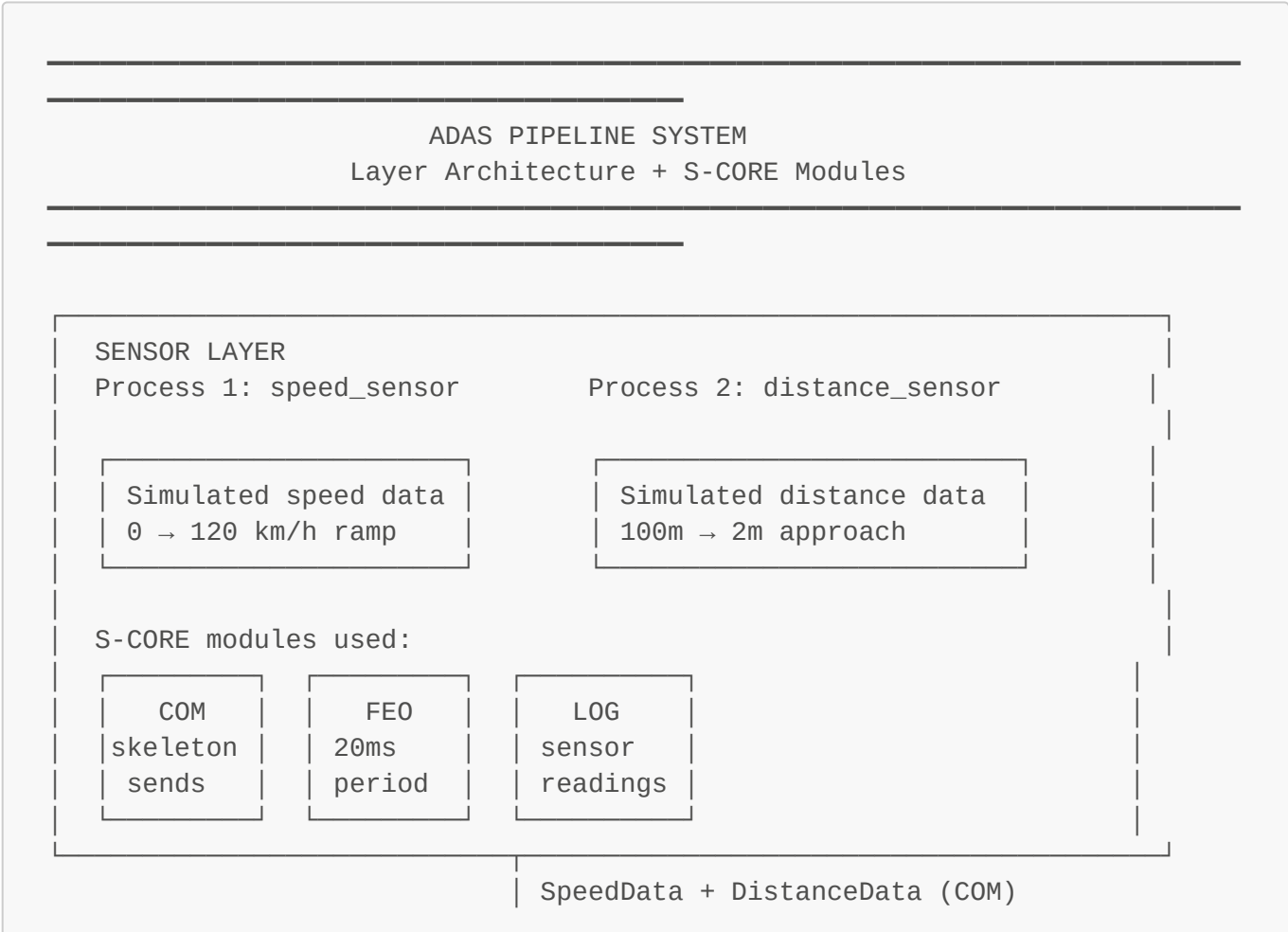


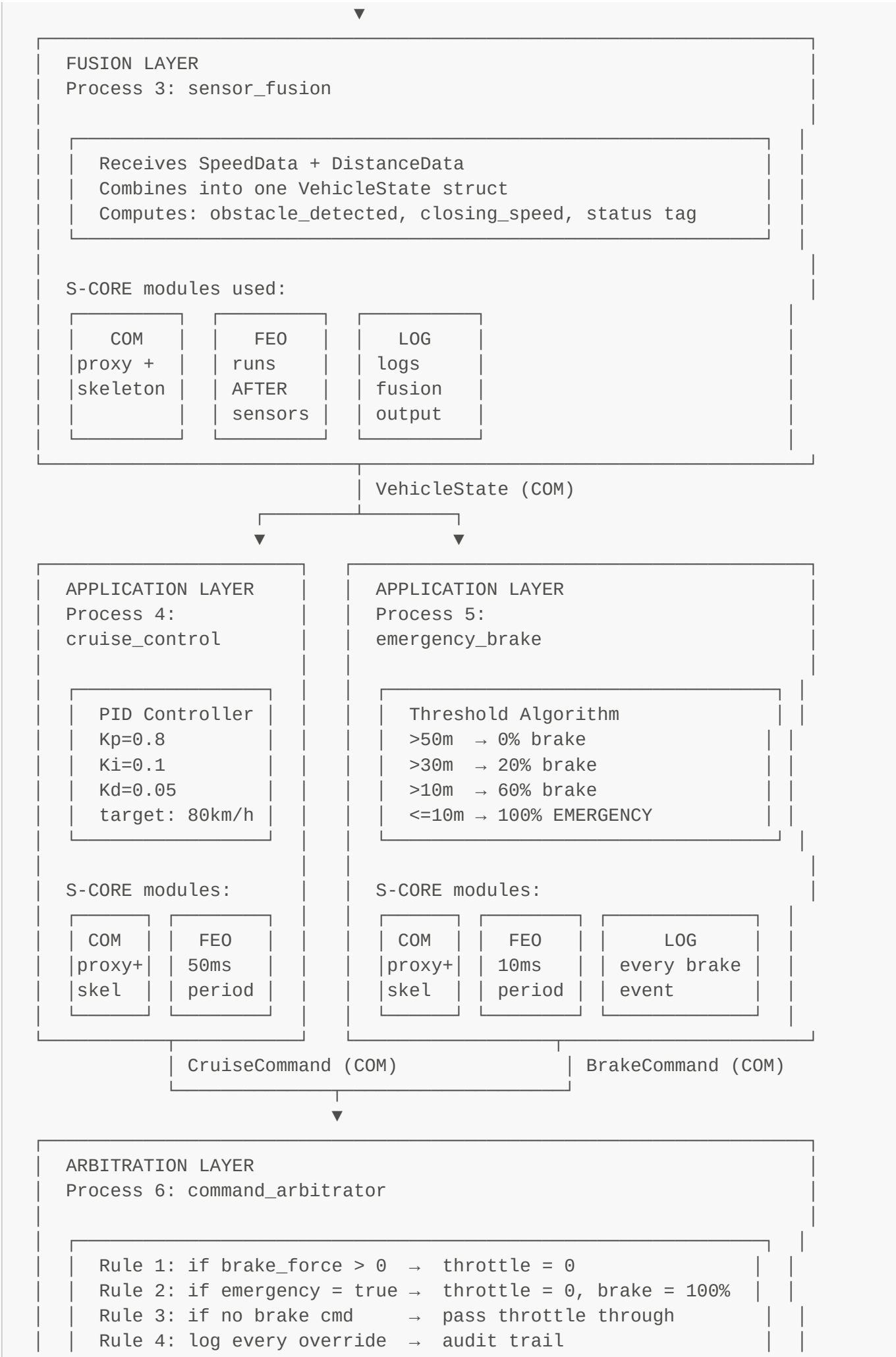
HARDWARE LAYER (simulated actuators)

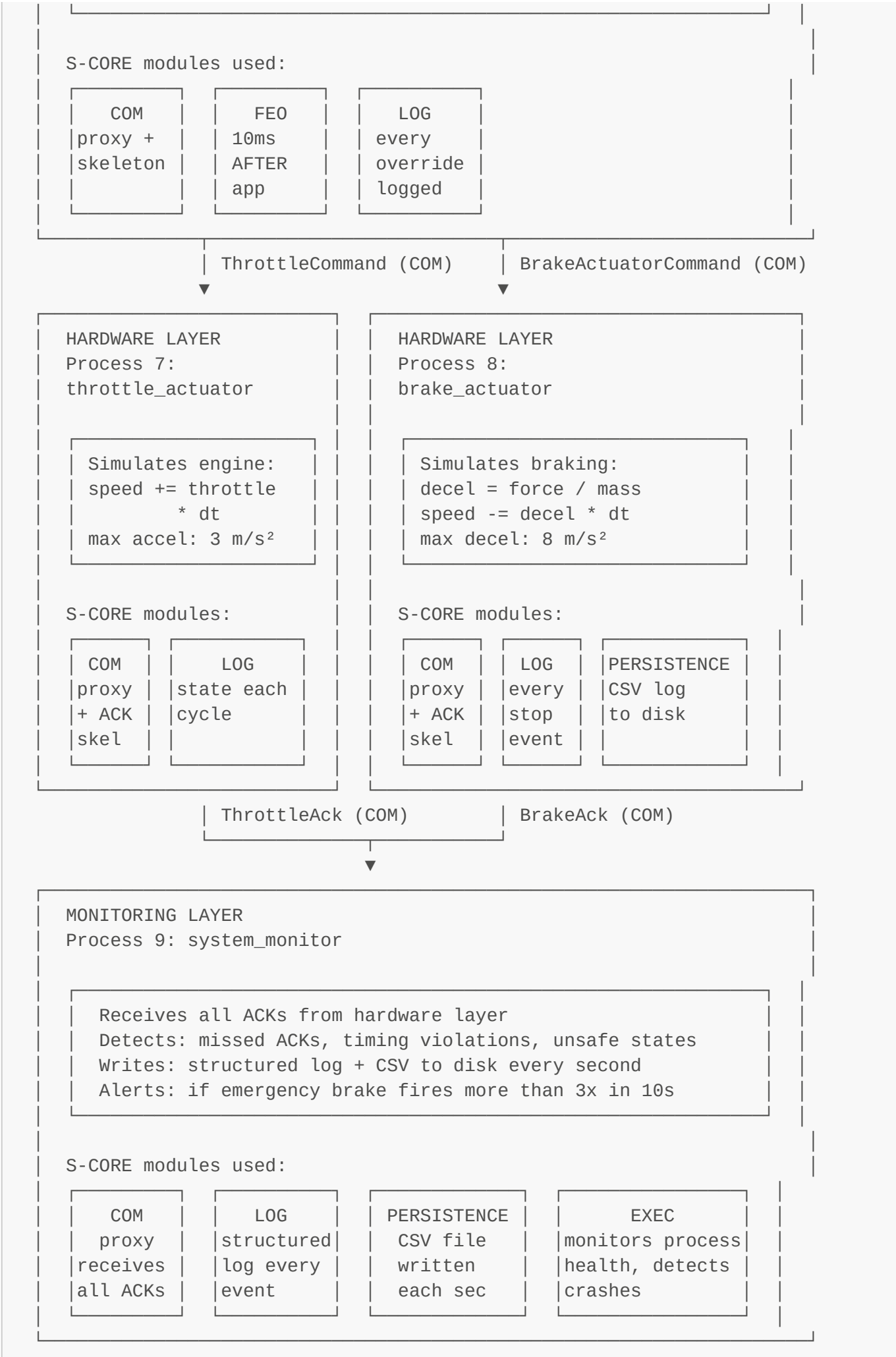


9. Layer diagram with modules

In the following diagram we have the different layers of our system, and most importantly each s-core module that might be used in this each layer :







MODULE SUMMARY

COM → every process (data exchange between all layers)
 FEO → sensor, fusion, app, arbitration (timing control)
 LOG → fusion, app, arbitration, hardware (event recording)
 PERSISTENCE → brake_actuator, monitor (disk storage)
 EXEC → monitor only (process health supervision)

10. Project File Structure

```

adas_pipeline/
├── src/
│   ├── datatype.h                ← all structs + interfaces
│   ├── processes/
│   │   ├── speed_sensor.cpp
│   │   ├── distance_sensor.cpp
│   │   ├── sensor_fusion.cpp
│   │   ├── cruise_control.cpp
│   │   ├── emergency_brake.cpp
│   │   ├── command_arbitrator.cpp
│   │   ├── throttle_actuator.cpp
│   │   ├── brake_actuator.cpp
│   │   └── system_monitor.cpp
│   ├── algorithms/
│   │   ├── pid_controller.h      ← reusable PID class
│   │   └── pid_controller.cpp
│   └── etc/
│       └── mw_com_config.json
├── BUILD
└── src/BUILD
  
```

Each process gets its own `.cpp` file with its own `main()` — they compile into separate executables, just like real separate ECU software.

11. The Simulated Scenarios

The sensor process will cycle through three realistic scenarios automatically:

Scenario 1 – Normal cruise (0s to 10s):
 speed: ramps from 0 to 80 km/h smoothly
 distance: stays at 100m (no obstacle)
 expected: cruise control maintains 80 km/h, no braking

Scenario 2 – Obstacle approaching (10s to 20s):

speed: holds at 80 km/h

distance: decreases from 100m to 25m (obstacle ahead)

expected: cruise control reduces throttle, brake system activates at 30m

Scenario 3 – Emergency (20s to 25s):

speed: still 80 km/h

distance: drops from 25m to 5m rapidly

expected: emergency brake fires, throttle cut to 0, full brake applied
